Lab 3 – Architecture Selection & Justification

Automated Rubric Assignment Evaluator

Name: Nagalalpuram Sai Mukesh Reddy
SRN: PES1UG24AM170
Section: 5C

Selected Architecture

A Microservices Architecture was selected for the Automated Rubric Assignment Evaluator, specifically the Peer Review Allocation & Moderation workflow. The system is organized into independently defined services: Reviewer Allocation, Review Management, Score Aggregation & Moderation, and Notification. Faculty and student interfaces access the relevant services through HTTPS/JSON APIs. The services use the Review Database Service for submissions, assignments, rubric scores and reviews, and the Audit Log Service for allocation and moderation records.

Why Microservices Architecture Fits This Scenario

1. Independent scalability. Reviewer allocation and score aggregation can experience heavy workloads when many submissions arrive near a deadline. Keeping these responsibilities in separate services allows the busiest service to be scaled independently, rather than scaling the entire application.

2. Independent deployment and maintenance. Reviewer allocation rules, review validation, score aggregation, and notification logic have distinct responsibilities. A change to the allocation algorithm can be developed and deployed without requiring the review-submission or notification service to be redeployed. This supports the system's evolving peer-review and moderation requirements.

Security Advantage

Each service can enforce role-based authorization and expose only the operations required by its consumers. Reviewer and author identities must be hidden from student-facing responses according to the configured anonymity mode, while identity resolution and moderation operations are restricted to authorized faculty. HTTPS protects service communication, and moderation and allocation events are recorded in a protected, append-only audit log.

Performance Benefit

Reviewer allocation and score aggregation can execute independently of the user interfaces, and reviewer notifications can be processed asynchronously. This reduces the need to keep a user request waiting for notification delivery and allows allocation workloads to be scaled separately. The implementation must still manage API latency, service failures, and consistency of assignment data.